

14. Governance / Guardrails / Ops

Reliability Specification

Risk controls, publication safety, incident handling, and operational trust model

Project: QuantPipeline

Document status: Working baseline for product and engineering delivery

Version: v1.0

Date: 2026-04-23

Audience: Product owner, architecture, development, QA, and future implementation agents

Purpose: This document is intended to be implementable. It defines contracts, boundaries, and decisions that should survive future feature growth without breaking the core recommendation pipeline.

Document Control

Field	Value
Document code	14
Title	Governance / Guardrails / Ops Reliability Specification
Primary input set	Documents 01-09, A1 technology recommendation, and baseline truth table
Primary architecture anchor	Data & Feature Layer -> Signals -> Selection -> Allocation -> Timing -> Risk Overlay -> Recommendation Object -> Backtest / Paper / Replay / Ops
Assumed release posture	Private single-user first release with future-safe modular boundaries
Key dependencies	Documents 10-13, 08 PRD, 09 Functional Requirements
Quality rule	No silent degradation, explicit trust semantics, and replayable output

1. Purpose and Governance Intent

This document defines the controls that keep QuantPipeline honest and reliable. It covers governance roles, decision guardrails, publication and suppression logic, operational reliability targets, incident handling, security, privacy, and audit evidence requirements.

The intent is not bureaucratic overhead. The intent is to prevent a private decision platform from presenting false confidence. Governance here means that recommendations are published, suppressed, changed, and reviewed under explicit rules rather than hidden ad hoc behavior.

2. Control Model Overview

Governance, Guardrails, and Reliability Controls

Document 14 anchor diagram

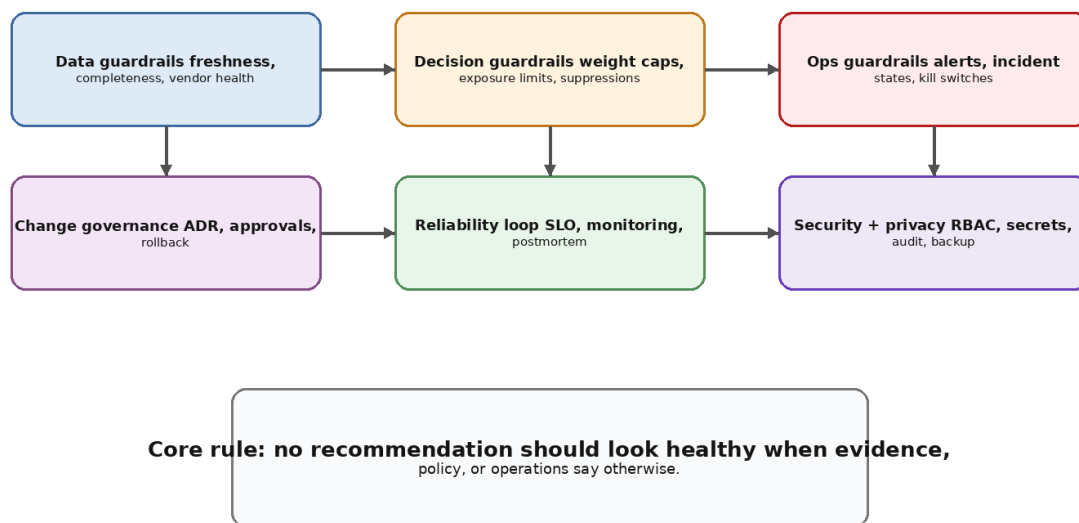


Figure 14-1. Governance, guardrails, and reliability control model.

3. Governance Principles

- No recommendation should look healthy when evidence, policy, or operations say otherwise.
- Publication status is a governed state, not a UI formatting choice.
- Guardrails should be codified and testable.
- Overrides are allowed only when explicit, auditable, and time-bounded.
- Methodology changes require named ownership and review.
- Security and privacy controls apply even in a private single-user product.
- Reliability is measured both technically and semantically.
- Incidents and postmortems should change the system, not just produce notes.

4. Roles and Responsibilities

The first release may be operated by one primary owner, but the governance model should still distinguish responsibilities. This prevents loss of clarity when the same person is acting as viewer, operator, approver, and system maintainer.

Role	Primary responsibilities	Typical actions
Viewer	Consume recommendations and validation outputs	Read only
Operator	Monitor health, manage incidents, inspect failures	Acknowledge alerts, open incidents, run diagnostics
Approver	Approve or suppress publication under defined	Approve staged recommendation, suppress

Role	Primary responsibilities	Typical actions
	rules	publication
Administrator	Manage policies, permissions, and control settings	Promote policy bundle, change guardrail thresholds
Maintainer	Maintain infrastructure, secrets, and deployment pipelines	Rotate secrets, deploy fixes, restore backups

5. Guardrail Taxonomy

Guardrails should be grouped by what they protect. This helps both implementation and user explanation.

Guardrail family	Protects against	Examples
Data guardrails	Stale, incomplete, or contradictory evidence	Freshness max age, missing-source threshold
Decision guardrails	Unsafe or incoherent portfolio proposals	Max position weight, sector cap, cash floor
Publication guardrails	User-facing false normalcy	Suppress if critical incident is active
Operational guardrails	Silent system failure	Queue lag threshold, failed job threshold
Security guardrails	Unauthorized access or secret exposure	RBAC, audit, secret rotation
Change guardrails	Unreviewed drift in methodology or policy	Approval required for bundle promotion

6. Data Guardrails

Data guardrails determine whether a recommendation can be trusted enough to publish. They should be evaluated before and after feature and signal generation, because some issues appear only after fusion.

Guardrail outcomes should be stored as machine-readable objects so admin surfaces and publication logic consume the same result.

Rule type	Example	Typical consequence
Freshness	News feed older than approved threshold	Degraded or suppressed publication
Completeness	Feature coverage below minimum asset threshold	Block selection or mark degraded
Consistency	Major timestamp misalignment across sources	Raise incident and suppress publication
Vendor health	Repeated upstream failures	Fallback if approved; otherwise suppress
Reference integrity	Unresolved asset mapping on critical inputs	Reject affected records and alert operator

7. Decision and Portfolio Guardrails

Decision guardrails should apply after allocation and timing, and again after risk overlay. The risk overlay is itself a control stage, but additional publication checks may still be needed.

Guardrail	Example policy	Stored evidence
Max single position	No final target weight above approved ceiling	Before and after weights, rule trigger
Sector concentration	Sector exposure cap per taxonomy	Sector exposure snapshot
Gross exposure	Maximum gross exposure after timing and risk overlay	Portfolio exposure metrics
Cash floor	Minimum cash in stress regime	Regime tag and final cash level
Suppression rule	Do not publish if critical incident open	Incident reference and publication reason

8. Publication States and Suppression Logic

Publication is a finite-state workflow. Recommended baseline states: draft, staged_for_approval, published, published_with_warning, suppressed, stale, and retired. The overview surface should reflect these states directly rather than inferring them from background signals.

Suppression should be used when a recommendation should not be treated as current guidance. This may be because a critical incident exists, a mandatory data guardrail failed, or a manual governance action intentionally blocked publication.

State	Meaning	Allowed transitions
draft	Run completed but not yet staged	staged_for_approval, suppressed
staged_for_approval	Awaiting explicit decision	published, published_with_warning, suppressed
published	Current approved recommendation	retired, stale, suppressed
published_with_warning	Published but marked degraded	retired, stale, suppressed
suppressed	Not fit to serve as current guidance	draft after rerun, retired
stale	Previously published but beyond freshness policy	published after refresh, suppressed, retired

9. Reliability Model and SLOs

Reliability should be expressed in both service-level and product-level terms. Uptime alone is not enough. The platform also needs freshness, queue, and publication quality indicators.

SLI family	Example SLI	Suggested target
API responsiveness	Overview p95 latency	Within interactive threshold

SLI family	Example SLI	Suggested target
Job success	Recommendation run success rate	High and stable
Freshness	Age of current recommendation inputs	Within approved envelope
Publication integrity	Percent of current recommendations with unresolved critical warnings	Zero
Admin visibility	Alert to acknowledgement time	Bounded by severity
Recovery	Mean time to restore healthy publication path after incident	Explicit target by severity

10. Monitoring and Alerting

Alerting must map to actionable operators and objects. Generic noise harms trust. Every alert should reference what is affected, what user-visible consequence exists or may occur, and what the expected next action is.

Recommended alert severities: informational, warning, high, and critical. Critical alerts should generally be publication-affecting unless explicitly exempted.

Alert family	Examples	Default owner
Source freshness	Market bars stale, macro release missing	Operator
Feature integrity	Feature job failed or completeness below threshold	Operator
Policy execution	Selection empty unexpectedly, risk overlay failure	Operator / approver
Publication workflow	Approval queue blocked or stale recommendation active	Approver
Security	Auth anomaly, secret rotation overdue	Administrator / maintainer
Platform	Queue lag, storage degradation, backup failure	Maintainer

11. Incident Management

Incidents should be used when alert-level handling is insufficient or when user trust may be affected. Incidents need severity, owner, timeline, linked objects, mitigation steps, and a final postmortem summary.

Publication-impacting incidents should automatically influence overview and admin state so the user is never shown a clean recommendation while a critical unresolved issue exists.

Severity	Typical impact	Expected workflow
SEV-1	Current recommendation	Immediate suppression or

Severity	Typical impact	Expected workflow
	cannot be trusted	warning, active incident room, postmortem required
SEV-2	Material degradation or partial publication risk	Rapid investigation, mitigation, explicit user-visible warning if published
SEV-3	Operational issue with limited immediate user harm	Owner assigned, bounded remediation plan
SEV-4	Minor issue or housekeeping	Tracked in backlog or routine ops

12. Security, Privacy, and Access Control

Even as a private product, QuantPipeline handles sensitive portfolio context, recommendation history, provider credentials, and decision metadata. Security must therefore be designed, not implied.

Minimum controls include role-based access, strong authentication, encrypted transport, server-side secrets, backup protection, immutable audit logs for high-risk actions, and explicit separation between read, simulate, approve, and admin mutation scopes.

13. Change Management and Promotion Control

Policy bundles, methodology versions, model artifacts, and major guardrail changes should move through a documented promotion workflow. Promotion records must state what changed, why it changed, which evidence package justified it, and what rollback path exists.

A bundle should not be promoted directly from experimentation into publication without validation evidence, approval, and a recorded effective time.

Change type	Approval expectation	Rollback expectation
Policy bundle promotion	Formal approval	Previous bundle remains restorable
Methodology version change	Formal approval	Historical reports preserve prior methodology labels
Guardrail threshold change	Formal approval if publication-impacting	Old threshold retained for rollback
UI-only explanatory change	Lightweight review	Standard release rollback
Secret or credential rotation	Operational approval	Verified fallback procedure

14. Backup, Recovery, and Continuity

The platform needs recoverability for canonical relational state, object manifests, policy artifacts, audit logs, and critical reports. Recovery plans should distinguish between restoring service availability and restoring trustworthy publication.

After a restoration event, a recommendation should not be auto-published until freshness and guardrails are re-evaluated.

Asset	Protection rule	Recovery note
PostgreSQL domain data	Regular backups and restore drills	Verify recommendation and audit consistency after restore
Object storage manifests and artifacts	Versioning or replicated storage	Preserve checksums and pointers
Policy bundles and model artifacts	Immutable versioned package store	Promotion history must remain intact
Secrets and credentials	Managed secret store and rotation	Restore requires controlled re-issuance
Monitoring configuration	Version controlled	Needed to restore visibility quickly

15. Audit and Evidence Requirements

Audit is not only for security actions. QuantPipeline should preserve evidence for publication decisions, suppressions, policy promotions, overrides, and methodology exceptions. This evidence is essential for replay and for later trust repair if the system behaves unexpectedly.

Action	Minimum audit fields
Publication approval	actor, object_ref, expected_state, decision_note, occurred_at
Suppression	actor, object_ref, reason_code, linked_incident, occurred_at
Policy promotion	actor, policy_version_id, evidence_package_ref, occurred_at
Manual override	actor, target_object, override_type, expiry, reason
Methodology exception	actor, methodology_version, failed_rule, justification, review_expiry

Appendix A - Core Runbooks

- Runbook 1: stale source blocks publication.
- Runbook 2: recommendation run fails after signal generation.
- Runbook 3: critical incident while a recommendation is currently published.
- Runbook 4: rollback to previous policy bundle after post-promotion issue.
- Runbook 5: restore from backup and re-establish trustworthy publication state.

Appendix B - Governance Questions for Every Promotion

1. What changed in the policy or methodology bundle?
2. What evidence package supports the change?
3. What new risks were introduced or reduced?
4. How will rollback be performed if the change causes operational or trust issues?
5. What user-visible status or explanation changes are required?